

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

по курсу «Операционные системы»

на тему:

«Буферизованный и небуферизованный ввод-вывод»

Студент ИУ7-62Б (Группа) _____ (Подпись, дата) Жаринов М. А. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Рязанова Н. Ю. (И. О. Фамилия)

2026 г.

1 Структуры

```
struct _IO_FILE
{
    int _flags;     /* High-order word is _IO_MAGIC; rest is flags. */

    /* The following pointers correspond to the C++ streambuf
       protocol. */
    char *_IO_read_ptr; /* Current read pointer */
    char *_IO_read_end; /* End of get area. */
    char *_IO_read_base;    /* Start of putback+get area. */
    char *_IO_write_base;   /* Start of put area. */
    char *_IO_write_ptr;    /* Current put pointer. */
    char *_IO_write_end;    /* End of put area. */
    char *_IO_buf_base; /* Start of reserve area. */
    char *_IO_buf_end;  /* End of reserve area. */

    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area.
       */
    char *_IO_backup_base;  /* Pointer to first valid character of
       backup area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
    int _flags2:24;
    /* Fallback buffer to use when malloc fails to allocate one.  */
    char _short_backupbuf[1];
    __off_t _old_offset; /* This used to be _offset but it's too
       small.  */

    /* 1+column number of pbase(); 0 is unknown. */
    unsigned short _cur_column;
    signed char _vtable_offset;
    char _shortbuf[1];

    _IO_lock_t *_lock;
    __off64_t _offset;
    /* Wide character stream stuff.  */
    struct _IO_codecvt *_codecvt;
    struct _IO_wide_data *_wide_data;
    struct _IO_FILE *_freeres_list;
    void *_freeres_buf;
```

```
    struct _IO_FILE **_prevchain;
    int _mode;
    int _unused3;
    __uint64_t _total_written;
    /* Make sure we don't get into trouble again.  */
    char _unused2[12 * sizeof (int) - 5 * sizeof (void *)];
};
```

Листинг 1: Структура _IO_FILE

```
struct stat
{
    __dev_t st_dev;     /* Device.  */
    __ino_t st_ino;     /* File serial number.  */
    __nlink_t st_nlink;     /* Link count.  */
    __mode_t st_mode;       /* File mode.  */
    __uid_t st_uid;     /* User ID of the file's owner. */
    __gid_t st_gid;     /* Group ID of the file's group.*/
    int __pad0;
    __dev_t st_rdev;        /* Device number, if device.  */
    __off_t st_size;            /* Size of file, in bytes.  */
    __blksize_t st_blksize; /* Optimal block size for I/O.  */
    __blkcnt_t st_blocks;       /* Number 512-byte blocks allocated. */
    # ifdef __USE_XOPEN2K8
    /* Nanosecond resolution timestamps are stored in a format
    equivalent to 'struct timespec'.  This is the type used
    whenever possible but the Unix namespace rules do not allow the
    identifier 'timespec' to appear in the <sys/stat.h> header.
    Therefore we have to handle the use of this header in strictly
    standard-compliant sources special.  */
    struct timespec st_atim;        /* Time of last access.  */
    struct timespec st_mtim;        /* Time of last modification.  */
    struct timespec st_ctim;        /* Time of last status change.  */
    #  define st_atime st_atim.tv_sec   /* Backward compatibility.  */
    #  define st_mtime st_mtim.tv_sec
    #  define st_ctime st_ctim.tv_sec
    # else
    __time_t st_atime;          /* Time of last access.  */
    __syscall_ulong_t st_atimensec; /* Nscecs of last access.  */
    __time_t st_mtime;          /* Time of last modification.  */
    __syscall_ulong_t st_mtimensec; /* Nsecs of last modification.  */
    __time_t st_ctime;          /* Time of last status change.  */
    __syscall_ulong_t st_ctimensec; /* Nsecs of last status change.  */
    # endif
    __syscall_slong_t __glibc_reserved[3];
};
```

Листинг 2: Структура stat

2 Программы

2.1 Программа 1

```c
#include <fcntl.h>
#include <stdio.h>

int main() {
        FILE *fs1, *fs2;
        char buff1[20], buff2[20], c;
        int flag1 = 1, flag2 = 1;
        int fd = open("alphabet.txt", O_RDONLY);
        fs1 = fdopen(fd, "r");
        setvbuf(fs1, buff1, _IOFBF, 20);
        fs2 = fdopen(fd, "r");
        setvbuf(fs2, buff2, _IOFBF, 20);
        while (flag1 == 1 || flag2 == 1) {
                flag1 = fscanf(fs1, "%c", &c);
                if (flag1 == 1)
                fprintf(stdout, "%c", c);
                flag2 = fscanf(fs2, "%c", &c);
                if (flag2 == 1)
                fprintf(stdout, "%c", c);
            }
        fprintf(stdout, "\n");
        return 0;
    }
```

Листинг 3: Исходный код Программы 1

Скриншот результата выполнения программы 1 представлен на рисунке 1.

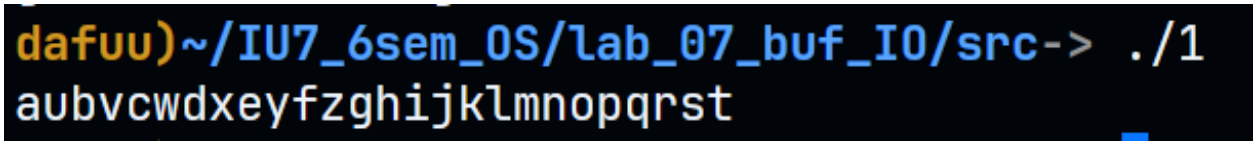


Рисунок 1 – Результат выполнения программы 1

Анализ полученного результата

Системный вызов open открывает файл alphabet.txt в режиме «только для чтения» (флаг O_RDONLY), создаёт структуру file. Указатель на эту структуру помещается в таблицу файловых дескрипторов процесса fd_array. Поскольку дескрипторы 0, 1 и 2 заняты стандартными потоками ввода-вывода, системный вызов возвращает наименьший свободный дескриптор (3), который присваивается переменной fd.

Библиотечной функции fdopen передается дескриптор fd и режим доступа «только для чтения». Функция возвращает указатель на инициализированную структуру FILE (fs1, fs2), где полю _fileno присваивается значение fd.

Функция setvbuf задаёт размер буфера 20 байт для fs1 и fs2.

Функция fscanf для fs1 сначала считывает 20 символов («a»—«t») из файла в буфер, при этом значение поля f_pos структуры file увеличивается на 20. Затем из буфера извлекается первый символ («a») и записывается в переменную c, а указатель _IO_read_ptr в структуре fs1 сдвигается на 1 байт. Вызов fprintf выводит этот символ в стандартный поток вывода.

Так как fs1 и fs2 ссылаются на один и тот же открытый файл, где значение f_pos равняется 20, то при вызове fscanf для fs2 в буфер считываются оставшиеся 6 символов («u»—«z») из файла. Первый символ из буфера fs2 («u») записывается в переменную c и выводится в стандартный поток вывода.

В последующих итерациях цикла поочерёдно будут выводиться символы из буферов buff1 и buff2. Когда будет достигнут конец буфера buff2, то программа продолжит выводить оставшиеся символы только из буфера buff1 (вплоть до t), пока не будет достигнут его конец.

На рисунке 2 отображены связи структур для программы 1.

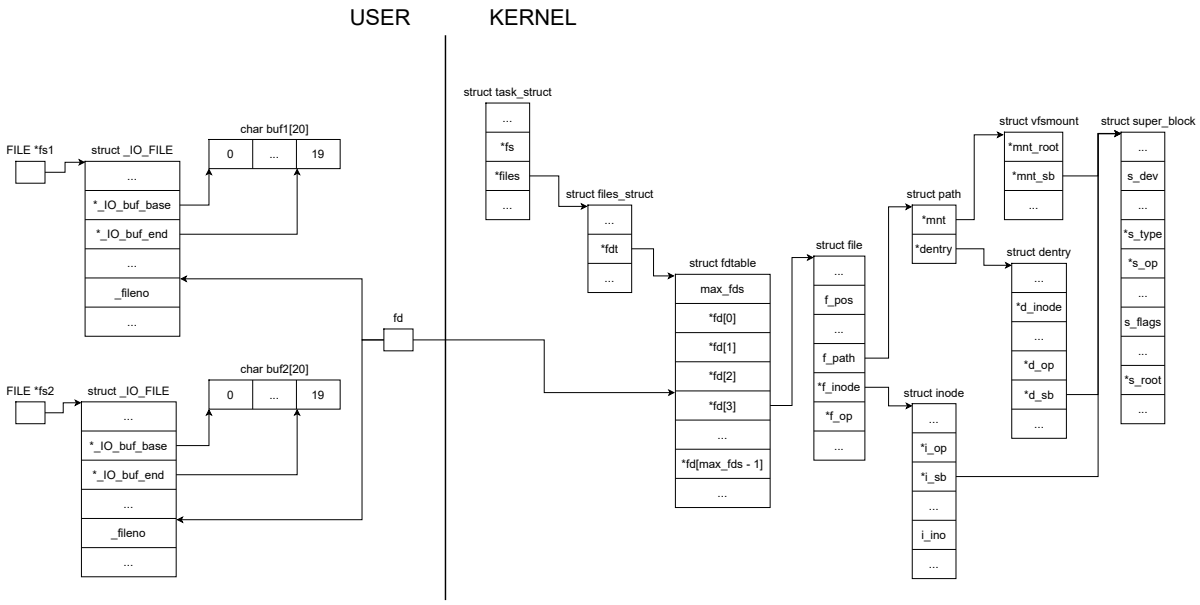


Рисунок 2 – Схема связи структур для программы 1

2.2 Программа 2

2.2.1 Один поток

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    char c;
    int flag1 = 1, flag2 = 1;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd1, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
            flag2 = read(fd2, &c, 1);
            if (flag2 == 1)
                write(1, &c, 1);
        }
    }
    close(fd1);
    close(fd2);
    c = '\n';
    write(1, &c, 1);
    return 0;
}
```

Листинг 4: Исходный код однопоточной Программы 2

Скриншот результата выполнения однопоточной программы 2 представлен на рисунке 3.

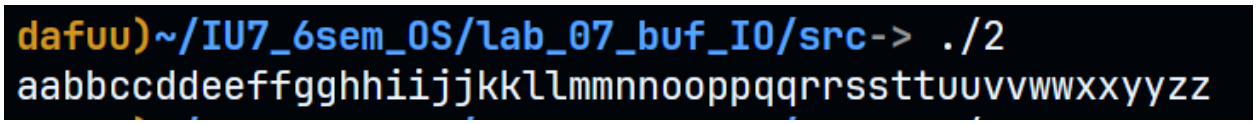


Рисунок 3 – Результат выполнения однопоточной программы 2

Анализ полученного результата

Системный вызов open создаёт два дескриптора открытого файла в режиме «только для чтения» (fd1, fd2). Соответственные дескрипторам структуры file ссылаются на один и тот же inode файла «alphabet.txt». Указатели на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4, так как дескрипторы 0,1 и 2 заняты стандартными потоками.

Системный вызов read для дескриптора fd1 считывает один символ («a»), что приводит к увеличению на 1 значения поля f_pos в соответствующей структуре file. Считанный символ выводится в стандартный поток вывода.

Так как дескрипторам fd1 и fd2 соответствуют независимые структуры file, то для fd2 значение поля f_pos равно 0. Поэтому при вызове read для fd2 будет считан тот же самый первый символ («a»), который также выведется в стандартный поток вывода.

Поочерёдное чтение и вывод символов будут продолжаться в цикле, пока значения f_pos в обеих структурах file не достигнут конца файла.

Таким образом, в стандартный поток вывода каждый символ алфавита будет выведен строго последовательно дважды (aabbcc...).

2.2.2 Один основной поток и один дополнительный поток

```
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>

int flag1 = 1, flag2 = 1;

void* thread1_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

int main() {
    char c;
    pthread_t t1, t2;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    pthread_create(&t1, NULL, thread1_func, &fd1);
    while (flag1 == 1 && flag2 == 1) {
        flag2 = read(fd2, &c, 1);
        if (flag2 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd2);
    pthread_join(t1, NULL);
    c = '\n';
    write(1, &c, 1);
    return 0;
}
```

Листинг 5: Исходный код Программы 2 с одним дополнительным потоком

Скриншот результата выполнения программы 2 с одним дополнительным потоком представлен на рисунке 4.

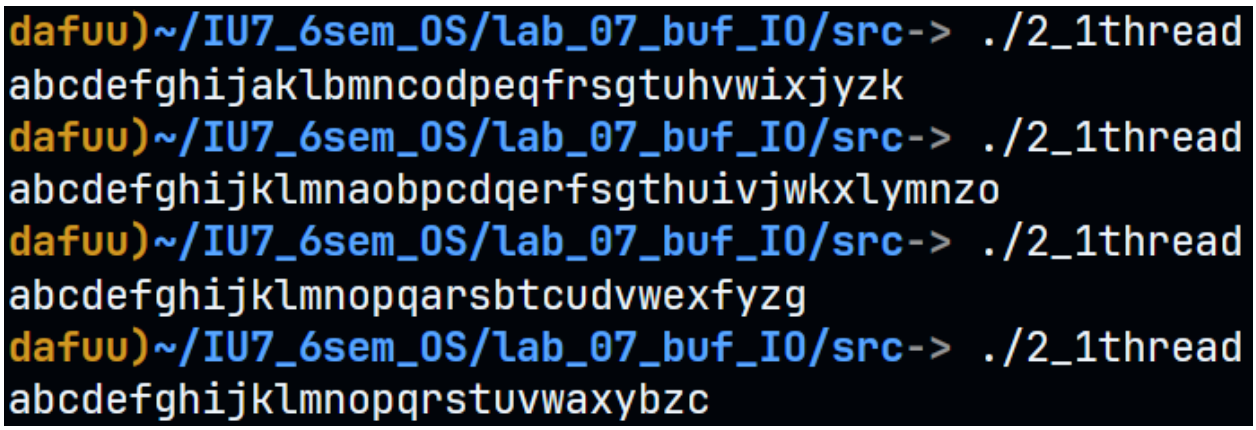


Рисунок 4 – Результат выполнения программы 2 с одним дополнительным потоком

Анализ полученного результата

Вызов pthread_create создаёт дополнительный поток, которому в качестве аргумента передаётся указатель на дескриптор fd1. Поскольку на создание потока требуется время, главный поток может успеть прочитать и вывести либо весь алфавит, либо его часть.

После создания дополнительного потока чтение и вывод символов осуществляются обоими потоками в зависящей от планировщика последовательности.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определён и зависит от того, как планировщик распределяет процессорное время между потоками.

2.2.3 Два дополнительных потока

```
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>

int flag1 = 1, flag2 = 1;

void* thread1_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

void* thread2_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag2 = read(fd, &c, 1);
        if (flag2 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

int main() {
    char c = '\n';
    pthread_t t1, t2;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    pthread_create(&t1, NULL, thread1_func, &fd1);
    pthread_create(&t2, NULL, thread2_func, &fd2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    write(1, &c, 1);
    return 0;
}
```

Листинг 6: Исходный код Программы 2 с двумя дополнительными потоками

Скриншот результата выполнения программы 2 с двумя дополнительными потоками представлен на рисунке 5.

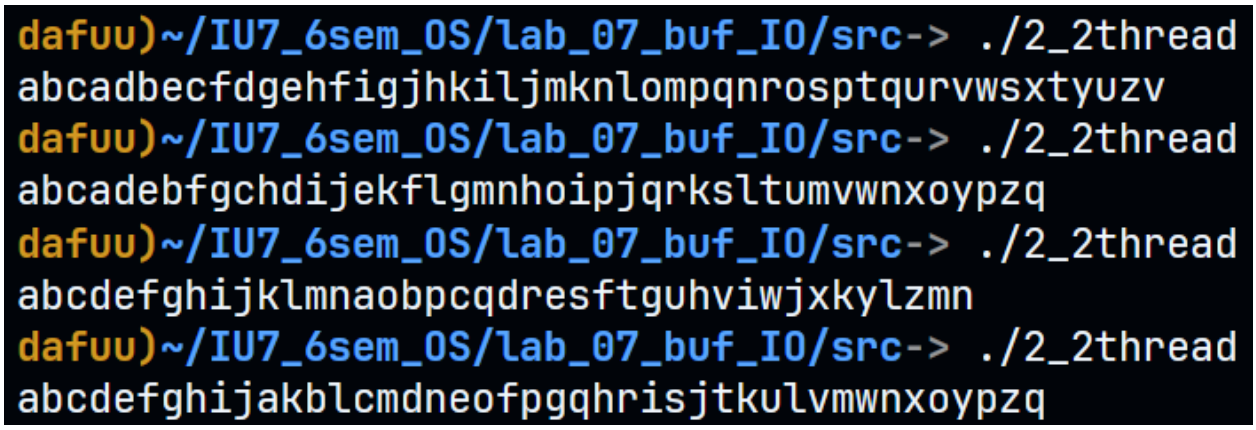


Рисунок 5 – Результат выполнения программы 2 с двумя дополнительными потоками

Анализ полученного результата

Два вызова pthread_create создают два дополнительных потока. Каждому из них передаётся указатель на свой файловый дескриптор.

На создание потоков требуется время и их порядок выполнения зависит от планировщика, поэтому порядок вывода символов неизвестен.

Таким образом, каждый символ файла будет напечатан дважды, однако порядок их вывода не определён и зависит от того, как планировщик распределяет процессорное время между дополнительными потоками.

На рисунке 6 отображены связи структур для программы 2.

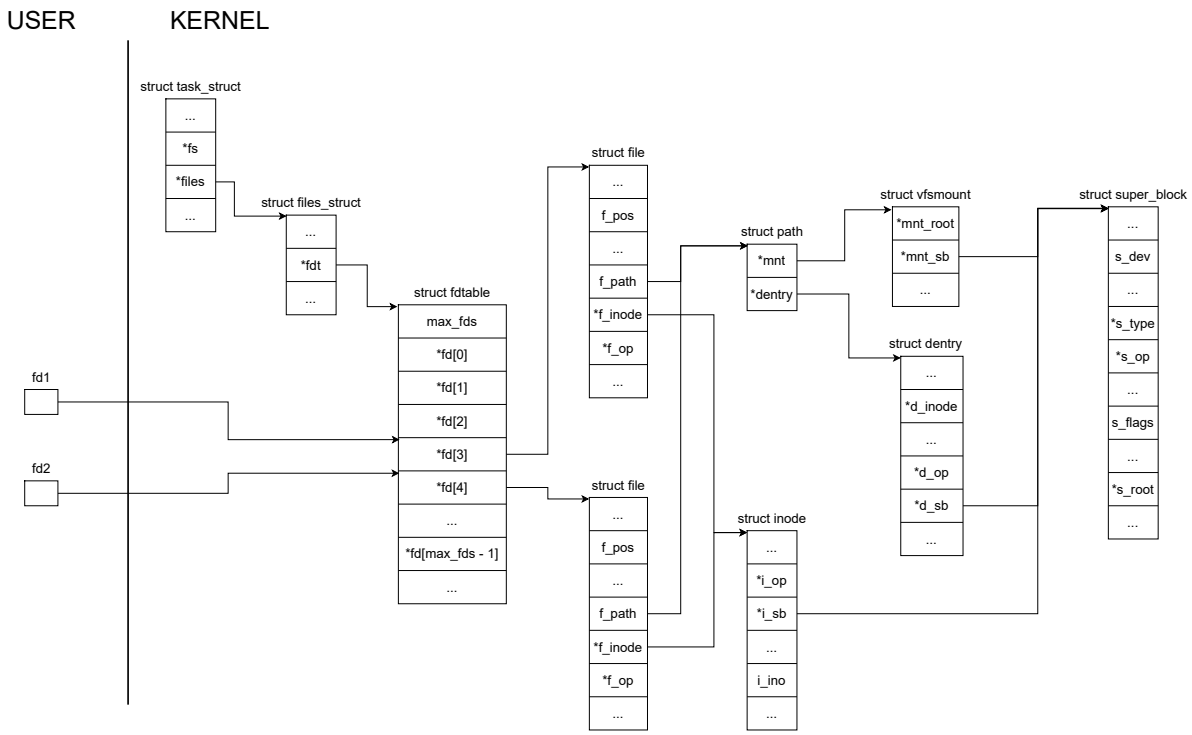


Рисунок 6 – Схема связи структур для программы 2

2.3 Программа 3

2.3.1 Небуферизованный ввод-вывод

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <stdio.h>

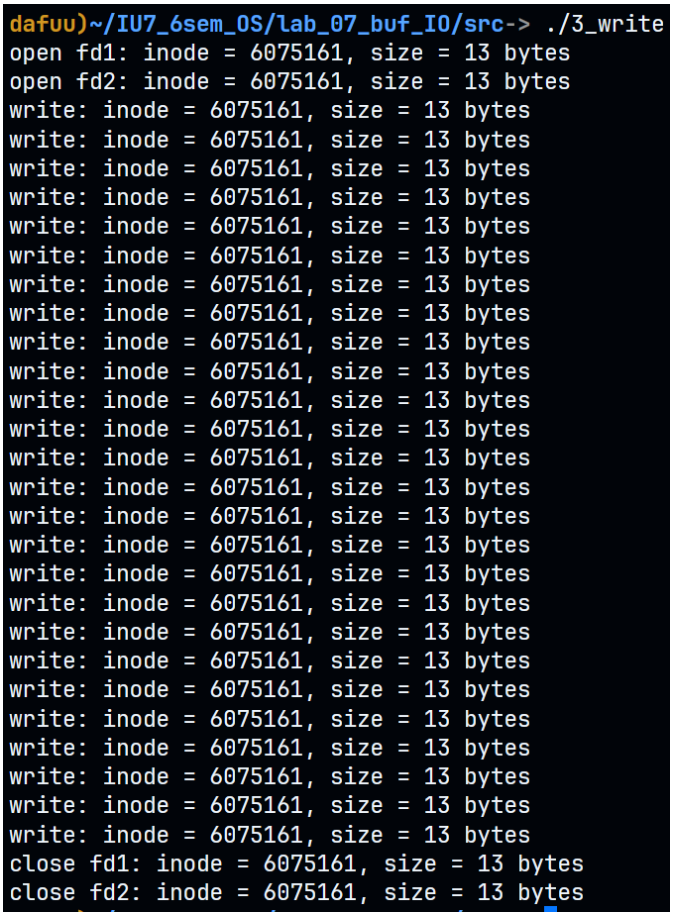
void print_stat(char *msg) {
    struct stat statbuf;
    stat("q.txt", &statbuf);
    fprintf(stdout, "%s: inode = %ld, size = %ld bytes\n", msg,
        statbuf.st_ino, statbuf.st_size);
}

int main() {
    int fd1 = open("q.txt", O_RDWR);
    print_stat("open fd1");
    int fd2 = open("q.txt", O_RDWR);
    print_stat("open fd2");
    for (char c = 'a'; c <= 'z'; c++) {
        if (c % 2)
            write(fd1, &c, 1);
```

```
20          else
21              write(fd2, &c, 1);
22          print_stat("write");
23      }
24      close(fd1);
25      print_stat("close fd1");
26      close(fd2);
27      print_stat("close fd2");
28      return 0;
29  }
```

Листинг 7: Исходный код Программы 3 с небуферизованным вводом-выводом

Скриншот результата выполнения программы 3 с небуферизованным вводом-выводом представлен на рисунке 7.


dafuu)~/IU7_6sem_OS/lab_07_buf_IO/src-> ./3_write
open fd1: inode = 6075161, size = 13 bytes
open fd2: inode = 6075161, size = 13 bytes
write: inode = 6075161, size = 13 bytes
write: inode = 6075161, size = 13 bytes
write

Анализ полученного результата

Системный вызов open создаёт два дескриптора открытого файла в режиме «чтения и записи» (O_RDWR). Соответствующие дескрипторам структуры file ссылаются на один и тот же inode файла «q.txt». Ссылки на эти структуры помещаются в таблицу открытых процессом файлов под индексами 3 и 4.

В первой итерации цикла системный вызов write для дескриптора fd1 записывает символ «a» в открытый файл, что приводит к увеличению на 1 значения поля f_pos в соответствующей структуре file. Размер файла становится равным 1 байту.

Так как дескрипторам fd1 и fd2 соответствуют независимые структуры file, то для fd2 значение поля f_pos остаётся равным 0. Поэтому на второй итерации цикла вызов write для fd2 заменит символ «a» в файле на символ «b». Размер файла остаётся равным 1 байту, а значение f_pos для fd2 увеличится на 1.

Поочерёдная запись символов в файл продолжается до конца цикла.

Таким образом, в файл будет записана только каждая вторая буква алфавита, начиная с «b», и итоговый размер файла составит 13 байт, так как вызов write для fd2 будет заменять символы, записанные вызовом write для fd1.

2.3.2 Буферизованный ввод-вывод

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <sys/stat.h>
4
5  void print_stat(char *msg) {
6      struct stat statbuf;
7      stat("q.txt", &statbuf);
8      fprintf(stdout, "%s: inode = %ld, size = %ld bytes\n", msg,
           statbuf.st_ino, statbuf.st_size);
9  }
10
11 int main() {
12     FILE *fs1 = fopen("q.txt", "w");
13     print_stat("fopen fs1");
14     FILE *fs2 = fopen("q.txt", "w");
15     print_stat("fopen fs2");
16     for (char c = 'a'; c <= 'z'; c++) {
```

```
17          if (c % 2)
18              fprintf(fs1, "%c", c);
19          else
20              fprintf(fs2, "%c", c);
21          print_stat("fprintf");
22      }
23      fclose(fs1);
24      print_stat("fclose fs1");
25      fclose(fs2);
26      print_stat("fclose fs2");
27      return 0;
28  }
```

Листинг 8: Исходный код Программы 3 с буферизованным вводом-выводом

Скриншот результата выполнения программы 3 с буферизованным вводом-выводом представлен на рисунке 8.

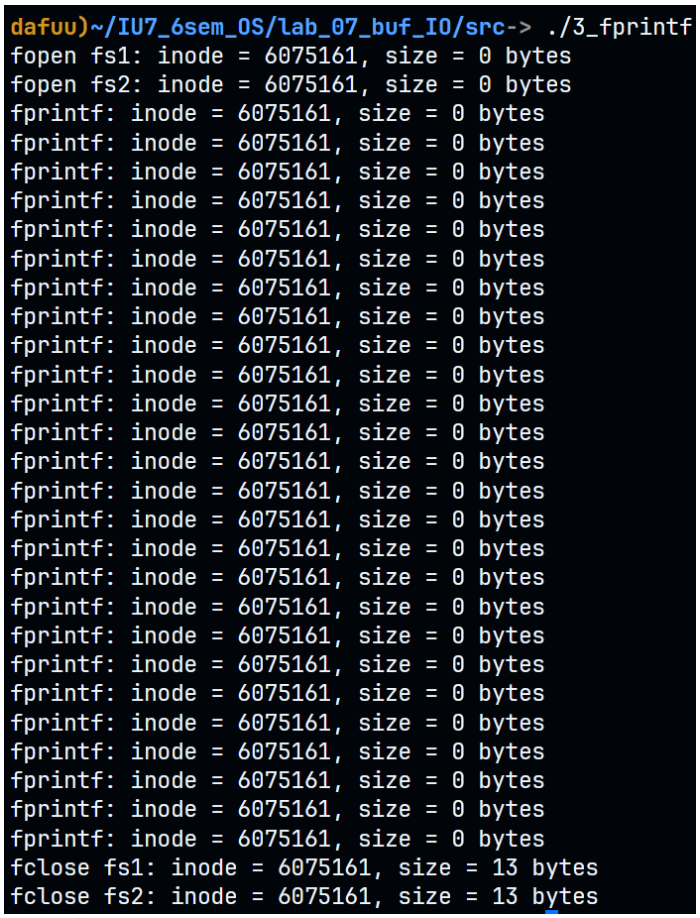


Рисунок 8 – Результат выполнения программы 3 с буферизованным вводом-выводом

Анализ полученного результата

Вызов библиотечной функции fopen откроет файл «q.txt» в режиме «только для записи» ("w") и вернёт два указателя на инициализированные структуры FILE (fs1, fs2). Во время вызова создаются две независимые структуры file с дескрипторами 3 и 4, значения которых будут записаны в поля _fileno соотвествующих структур _IO_FILE. Эти структуры file ссылаются на один и тот же inode файла «q.txt». Для каждой структуры FILE создается свой внутренний буфер в пространстве пользователя.

Вызов библиотечной функции fprintf для fs1 записывает символ «a» в буфер, что приводит к смещению в соответствующей структуре указателя _IO_WRITE_PTR на 1 байт.

Вызов библиотечной функции fprintf для fs2 записывает символ «b» в буфер, что приводит к смещению в соответствующей структуре указателя _IO_WRITE_PTR на 1 байт.

В результате работы цикла в буфере fs1 будет находиться каждая вторая буква алфавита, начиная с «a», а в fs2 — каждая вторая буква алфавита, начиная с «b».

Вызов fclose для fs1 приводит к записи из буфера в открытый файл. Значение поля f_pos для дескриптора 4 и размера файла становится 13. После этого дескриптор закрывается.

Так как были созданы две структуры открытых файлов, ссылающихся на один и тот же inode, а в структуре, соответствующей fs1, значение поля f_pos осталось равным 0, то при вызове fclose для fs1 буфер записывается в открытый файл с начала, что приводит к замене содержимого буфера fs1 на содержимое буфера fs2. Размер файла остаётся тем же (13 байт).

Таким образом, итоговое содержимое файла зависит от порядка вызовов fclose. В данном случае в файл будет записан каждый второй символ алфавита, начиная с буквы «b», а размер файла составит 13 байт.

На рисунке 9 отображены связи структур для программы 3 с буферизованным вводом-выводом.

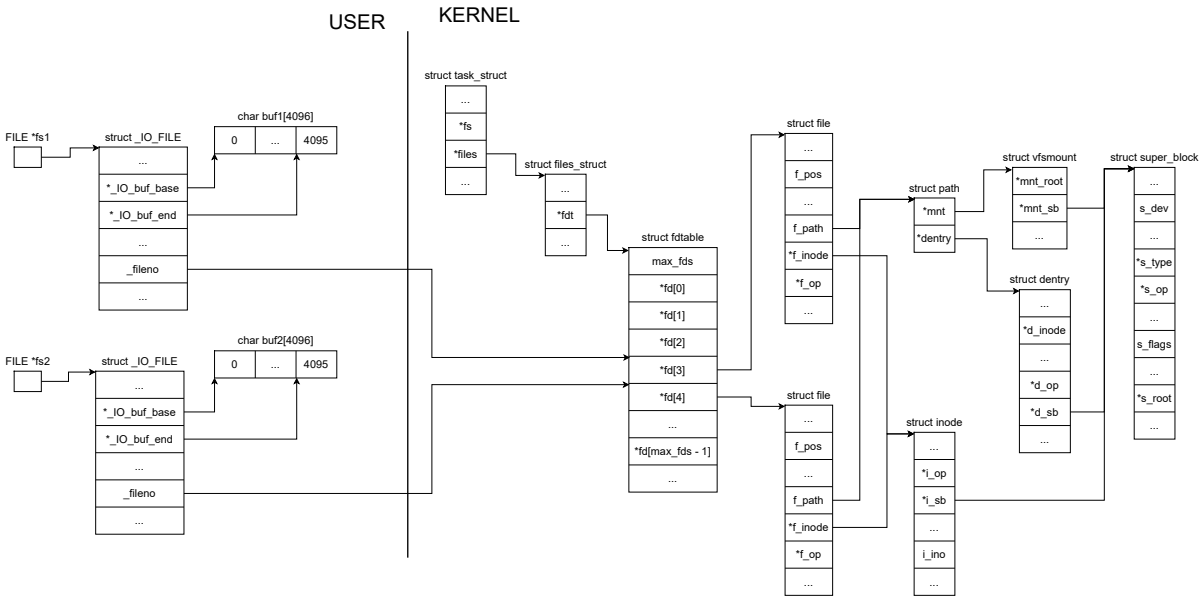


Рисунок 9 – Схема связи структур для программы 3 с буферизованным вводом-выводом